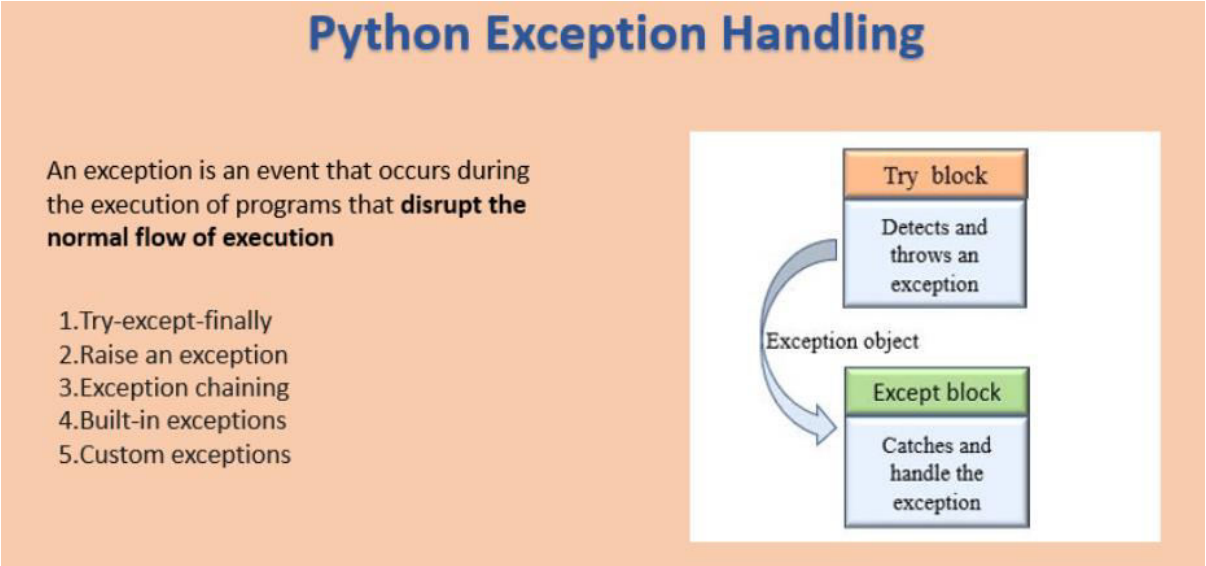
Error / Exception Handling

1.What is Error / Exception Handling?

Error / Exception Handling is the process of detecting, handling, and resolving errors in a program to prevent crashes and unexpected behavior. It ensures that the program can gracefully handle issues like invalid inputs, missing files, network failures, and more.



📌 Types of Errors

1. **Syntax Errors** – Occur due to incorrect code syntax.
2. **Runtime Errors (Exceptions)** – Happen during program execution, such as division by zero.
3. **Logical Errors** – The program runs but produces incorrect results.

---

2. Why is Error Handling Needed?

✓**Prevents Program Crashes** – Ensures the system continues running.
✓**Improves Debugging** – Helps identify the root cause of errors.
✓**Enhances User Experience** – Prevents abrupt failures.
✓**Ensures Data Integrity** – Prevents corrupted or lost data.
✓**Handles Unexpected Scenarios** – Manages errors like network failures or missing files.

---

3. Applications of Exception Handling

✅ **Web Development** – Handling invalid user input, authentication failures.
✅ **Database Management** – Managing failed transactions, connection issues.
✅ **Networking** – Handling timeouts, server unavailability.
✅ **Machine Learning** – Preventing crashes due to missing data or computation errors.
✅ **Financial Systems** – Ensuring failed transactions do not corrupt financial records.

4. Advantages of Exception Handling

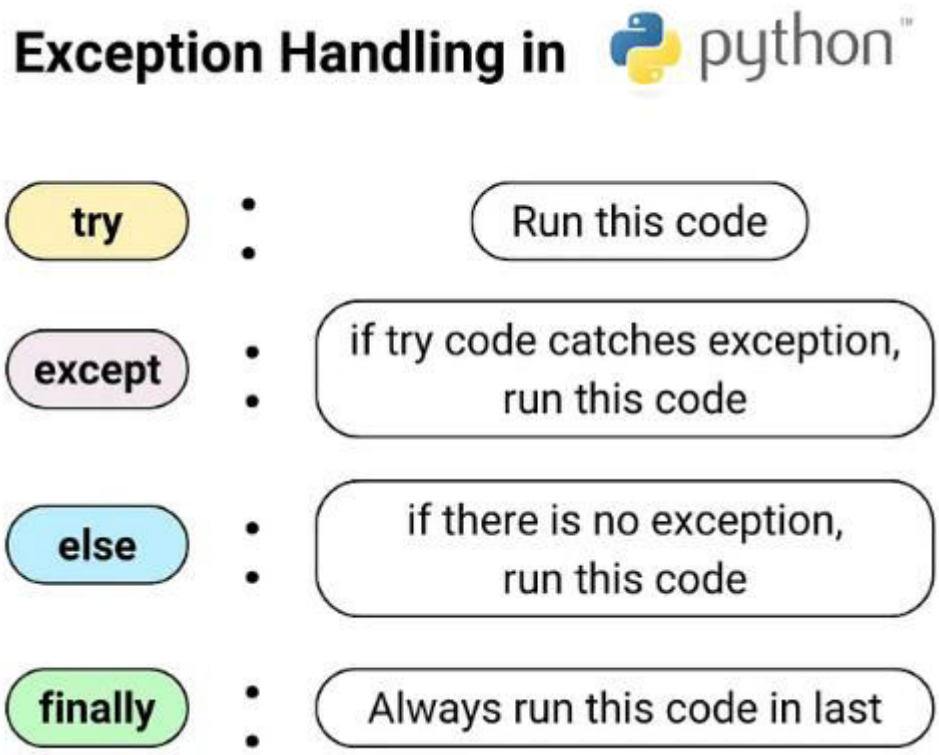
✅ **Maintains Program Flow** – The program doesn't crash unexpectedly.
✅ **Improves Readability** – Error messages help developers debug.
✅ **Ensures Data Consistency** – Transactions roll back if an error occurs.
✅ **Reduces Code Duplication** – One centralized error-handling mechanism.

5. Limitations of Exception Handling

❌ **Performance Overhead** – Handling exceptions slows down execution.
❌ **Complexity** – Too many try-catch blocks can make code messy.
❌ **Not a Fix for Poor Code** – Cannot replace good programming practices.
❌ **Hard to Debug Silent Failures** – Catching all exceptions may hide actual errors.

6. Real-World Applications

✅ **E-commerce Platforms (Amazon, Flipkart)** – Handling payment failures and invalid cart actions.
✅ **Banking Systems (PayPal, Stripe)** – Managing failed transactions and invalid credentials.
✅ **Operating Systems (Windows, Linux)** – Preventing crashes due to system errors.
✅ **Social Media (Facebook, Twitter)** – Handling API failures and user authentication errors.
✅ **Self-Driving Cars (Tesla, Waymo)** – Managing sensor failures and unexpected obstacles.



7. Key Points About Error Handling

✓**Always Handle Exceptions Properly** – Avoid silent failures.
✓**Use Logging** – Helps in debugging and monitoring.
✓**Avoid Overusing Try-Catch Blocks** – Use where necessary.
✓**Differentiate Between Expected & Unexpected Errors** – Handle known issues differently.
✓**Rollback Changes in Critical Transactions** – Prevents corruption of data.

8. Exception Handling Architecture

📌 **Key Components**

1. **Try Block** – Contains code that may raise an error.
2. **Except / Catch Block** – Handles the error.
3. **Finally Block** – Executes regardless of an error occurring.
4. **Raise / Throw** – Explicitly raises an exception.

🔹 Example: Python Exception Handling Architecture

python Copy Edit

```python
try:
    num = int(input("Enter a number: "))
    result = 10 / num  # May raise ZeroDivisionError
except ZeroDivisionError:
    print("Error: Cannot divide by zero!")
except ValueError:
    print("Error: Invalid input, please enter a number.")
finally:
    print("Execution completed.")
```

9. Interview Questions & Answers on Exception Handling

🟢 **Basic Level (For Beginners)**

1. **What is an Exception?**
Ans: An **exception** is an event that disrupts normal program execution, such as division by zero or file not found.

2. **How is exception handling implemented in Python?**
Ans: Using try, except, finally, and raise statements.

3. **What is the difference between an error and an exception?**
Ans:

- **Error**: Irrecoverable issues (e.g., syntax errors, memory overflow).
- **Exception**: Recoverable runtime issues (e.g., file not found).

🟡 **Intermediate Level (For Experienced Candidates)**

4. **What are Checked and Unchecked Exceptions in Java?**
Ans:

- **Checked Exceptions**: Compile-time exceptions (e.g., IOException).
- **Unchecked Exceptions**: Runtime exceptions (e.g., NullPointerException).

5. **What is the purpose of the finally block?**
Ans: The finally block executes **whether or not an exception occurs**, often used for closing database connections or file streams.

6. **How can exceptions be logged in a system?**
Ans: Using logging frameworks like log4j (Java) or logging module (Python) to store error details.

🔴 **Advanced Level (For Senior Developers & Architects)**

7. **How do you handle multiple exceptions in a single block?**
Ans: In Python, multiple exceptions can be handled like this:

python

CopyEdit

```
try:
    result = 10 / int(input("Enter a number: "))
except (ZeroDivisionError, ValueError) as e:
print(f"Error occurred: {e}")
```

8. **How does exception handling affect performance?**
Ans:

- Exception handling introduces **overhead** in execution.
- Using excessive try-catch blocks slows performance.
- Silent failures can make debugging harder.

9 **What are some best practices for handling exceptions?**
Ans:
✓**Use specific exceptions** instead of generic Exception.
✓**Log exceptions** for debugging.
✓**Avoid suppressing exceptions** without handling them.
✓**Use finally to clean up resources** like files and database connections.

🔟 **Conclusion: When to Use Exception Handling?**

✓**Always handle errors that can be predicted (e.g., invalid input).**
✓**Use exception handling in critical systems (e.g., finance, healthcare).**
✓**Don't rely on exceptions for normal control flow.**

Kaggle link

https://www.kaggle.com/code/mukulkirti/python-exception-handling

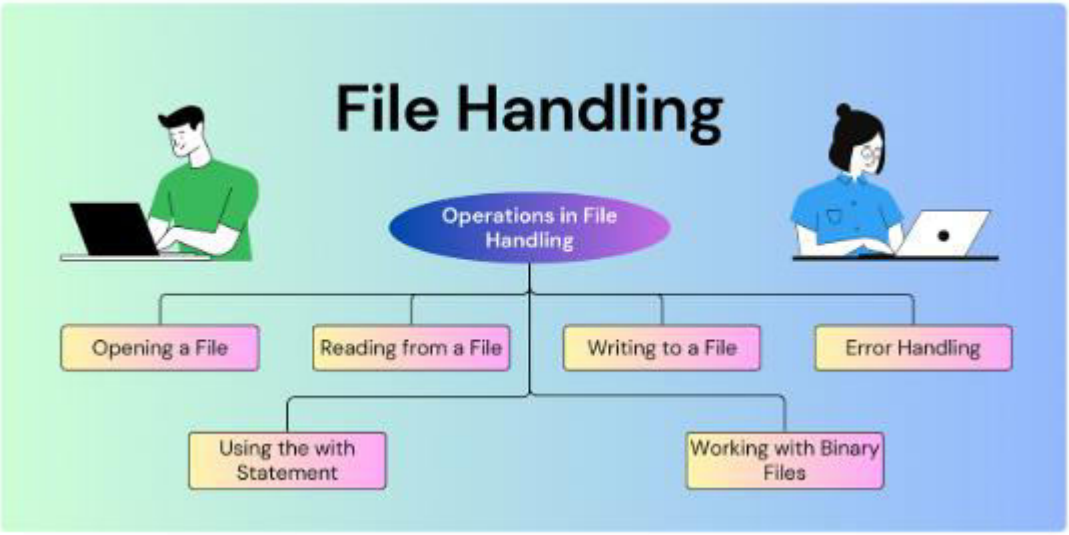
https://www.geeksforgeeks.org/python-exception-handling/

File Handlings

File Handling in Programming

What is File Handling?

File handling is a mechanism in programming that allows reading, writing, and manipulating files stored on a computer. It enables data storage and retrieval, making it essential for applications that require data persistence.



Why Use File Handling?

- To store data permanently (unlike variables that store data temporarily).

- To read, write, update, or delete data in a file.
- To handle large amounts of data efficiently.
- To enable data exchange between programs and systems.

Key Points of File Handling

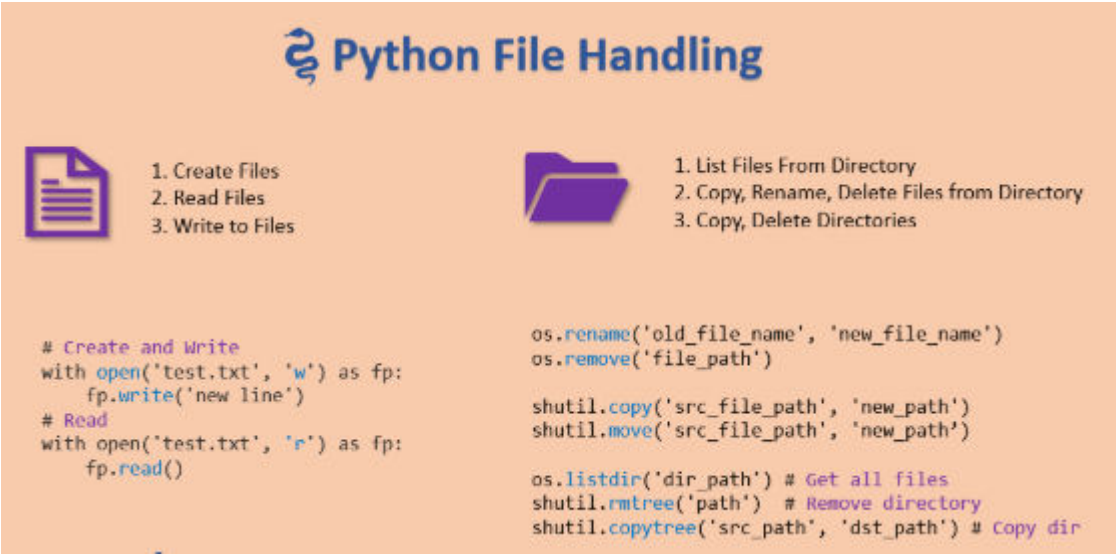
1. **File Operations**: Open, Read, Write, Append, Close, Delete.
2. **File Modes**: r (read), w (write), a (append), r+ (read & write), etc.
3. **File Types**: Text files (.txt, .csv) and Binary files (.bin, .dat).
4. **Error Handling**: Handling file-related errors using exceptions.
5. **Security**: Managing access permissions to prevent unauthorized modifications.

Examples of File Handling

Example 1: Writing to a File (Python)

python Copy Edit

```python
with open("example.txt", "w") as file:
    file.write("Hello, this is a file handling example.")
```



Explanation:

- open("example.txt", "w") opens the file in write mode (w).
- file.write("Hello...") writes data into the file.
- with open automatically closes the file after writing.

Example 2: Reading from a File

```python
with open("example.txt", "r") as file:
    content = file.read()
    print(content)
```

Output:

```kotlin
Hello, this is a file handling example.
```

Advantages of File Handling

✅ **Data Persistence**: Data is stored permanently.
✅ **Efficient Data Management**: Helps manage large amounts of data.
✅ **Data Sharing**: Files can be shared and accessed by multiple applications.
✅ **Backup & Recovery**: Important for system recovery.

Limitations of File Handling

❌ **Slow Access**: File handling is slower than databases.
❌ **Manual Handling**: Requires explicit read/write operations.
❌ **No Relationships**: Unlike databases, files do not support relations between datasets.
❌ **Security Risks**: Unauthorized access can lead to data loss or corruption.

Architecture of File Handling

1. **Application Layer**: User interacts with the file (via GUI/CLI).
2. **File System Interface**: Operating system provides APIs for file handling.
3. **Storage Layer**: Data is stored in HDD, SSD, or external storage.

Interview Questions & Answers on File Handling

Basic Level

1. **Q: What are different file handling modes in Python?**
   A: r (read), w (write), a (append), r+ (read & write), w+ (write & read), a+ (append & read).

2. **Q: How do you open and close a file in Python?**
   A: Using open() and close(). Example:

```python
file = open("data.txt", "r")
file.close()
```

3. **Q: What is the difference between text and binary files?**
   A:
   1. **Text files** store data as human-readable text (.txt, .csv).
   2. **Binary files** store data in a machine-readable format (.bin, .dat).

Intermediate Level

4. **Q: What happens if you try to open a file that does not exist in read mode?**
   A: A FileNotFoundError is raised.

5. **Q: How do you handle file errors in Python?**
   A: Using try-except:

```python
try:
    file = open("nonexistent.txt", "r")
except FileNotFoundError:
    print("File not found!")
```

6. **Q: How do you read a file line by line in Python?**
   A:

```python
with open("data.txt", "r") as file:
    for line in file:
        print(line.strip())
```

Advanced Level

7. **Q: How do you work with large files efficiently?**
   A: By reading files in chunks using .read(size):

```python
with open("large_file.txt", "r") as file:
    while chunk := file.read(1024):  # Read 1024 bytes at a time
        process(chunk)
```

Q: How do you serialize and deserialize objects in Python?
A: Using pickle module:

```python
import pickle
data = {"name": "John", "age": 25}
with open("data.pkl", "wb") as file:
    pickle.dump(data, file)  # Serialize
with open("data.pkl", "rb") as file:
    loaded_data = pickle.load(file)  # Deserialize
print(loaded_data)
```

Q: How do you append data to an existing file?
A: Using append mode (a):

```python
with open("data.txt", "a") as file:
    file.write("\nAppending new data.")
```

Docstrings

Docstrings in Python

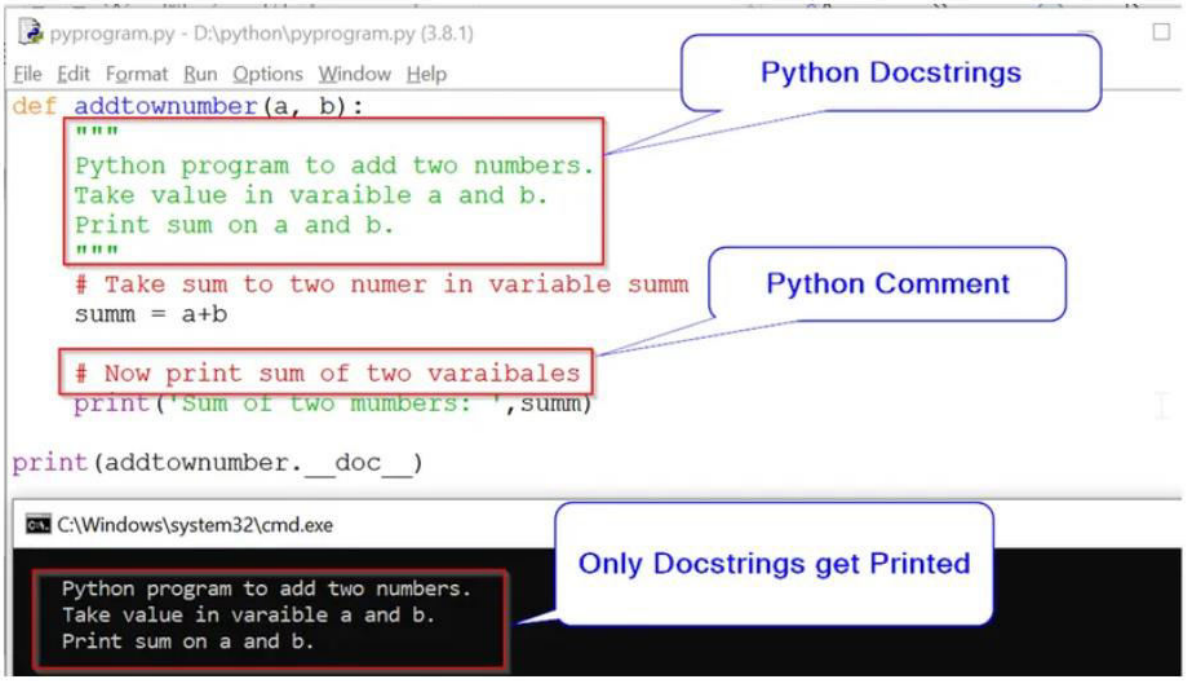
What is a Docstring?

A docstring (documentation string) is a special kind of comment enclosed in triple quotes (""" """ or ''' ''') that provides documentation for a module, function, class, or method in Python. It helps programmers understand the purpose and usage of code.

Example of Docstrings

```python
def add(a, b):
    """Returns the sum of two numbers."""
    return a + b

print(add.__doc__)  # Output: Returns the sum of two numbers.
```



Why Use Docstrings?

1. **Code Documentation:** Helps explain the functionality of code for future reference.
2. **Readability:** Improves code maintainability by providing explanations.
3. **Standardization:** Encourages consistency in documentation.
4. **Integration with Tools:** Works with help(), pydoc, and IDEs for automatic documentation.
5. **Onboarding New Developers:** Makes it easier for new team members to understand code quickly.

Key Points About Docstrings

✓ Docstrings are written using triple quotes (""" or ''').
✓ They are different from comments (#), as they are accessible at runtime using .__doc__.
✓ Docstrings should be written in a clear, concise, and structured format.
✓ They support multi-line descriptions.

Advantages of Docstrings

✅ **Enhances Code Readability:** Provides self-explanatory documentation.
✅ **Automates Documentation:** Used by tools like pydoc to generate documentation.
✅ **Improves Collaboration:** Helps teams work on the same codebase with better understanding.
✅ **Useful for API Development:** Helps users understand how to use functions and classes.

Limitations of Docstrings

❌ **No Enforced Format:** Unlike JavaDocs, Python doesn't enforce a strict format.
❌ **Can Be Ignored by Developers:** Some developers may not write or maintain docstrings properly.
❌ **Not a Substitute for Code Comments:** It explains "what" a function does but may not detail "why" it works a certain way.

Architecture of Docstrings

Python docstrings typically follow a structured format, especially in libraries and frameworks. A well-formatted docstring includes:

1. **Summary Line** – A brief description of what the function/class does.
2. **Extended Description** (Optional) – Additional details about parameters, return values, and exceptions.
3. **Parameter Descriptions** – Explanation of function arguments.
4. **Return Value Explanation** – What the function returns.
5. **Example Usage** (Optional) – Shows how to use the function.

Interview Questions on Docstrings

Basic Level Questions

1. **What is a docstring in Python?**
   Answer: A docstring is a string used to document a module, function, class, or method in Python. It is enclosed within triple quotes and can be accessed using .__doc__.
2. **How do you write a docstring for a function?**
   Answer: Use triple quotes (""" """ or ''' ''') inside the function and describe its purpose. Example:

```python
def greet():
    """This function prints a greeting message."""
    print("Hello, world!")
```

1. **What is the difference between a docstring and a comment?**
   Answer:
   - **Docstrings** are enclosed in triple quotes and can be accessed at runtime using .__doc__.
   - **Comments** (#) are ignored during execution and are only for developers.

Intermediate Level Questions

4. **How can you access a function's docstring?**
   Answer: Using the .__doc__ attribute or the help() function.

```python
def square(n):
    """Returns the square of a number."""
    return n * n

print(square.__doc__)  # Output: Returns the square of a number.
```

Links: https://www.kaggle.com/code/hagzilla/what-are-docstrings

4. **What are the best practices for writing docstrings?**
   Answer:
   - Keep them concise but informative.
   - Follow a consistent format.
   - Use a structured approach (summary, parameters, returns, exceptions).
   - Use tools like pydoc or Sphinx for generating documentation.

Advanced Level Questions

6. **What are different docstring formats used in Python?**
   Answer: The most common formats are:
   - **Google Style**
   - **NumPy Style**
   - **reStructuredText (RST) Style**

Example (Google Style):

```python
def multiply(x, y):
    """
    Multiplies two numbers.

    Args:
        x (int): The first number.
        y (int): The second number.

    Returns:
        int: The product of x and y.
    """
    return x * y
```

How can you generate documentation using docstrings?
Answer: Use Python's built-in pydoc module.

```bash
python -m pydoc <module_name>
```

Example: To generate documentation for a script `math_operations.py`:

```bash
python -m pydoc math_operations
```

6. **How do docstrings help in API development?**
   Answer: Docstrings provide clear documentation for API functions, allowing automatic generation of API documentation using tools like **Sphinx**, **Doxygen**, or **MkDocs**.